3-Week Plan: UART, SPI & I2C on a Programmable Protocol Emulator ASIC

Jane Street ASIC Competition (IHP 130nm via Tiny Tapeout, 6x4 tiles, ~24K logic cells) | Final deadline: 18 Jan 2027 | Goal: all three protocols running as **firmware** on one tiny programmable core, not as fixed hardware blocks

WEEK 1 (Days 1-7): Research, Architecture & Toolchain	
Research	• **Prior art:** RP2040 PIO (instruction set, FIFOs, side-set, clock divider) and TI PRU cores; note what you would do differently • **Protocols:** UART (start/data/parity/stop, oversampling), SPI modes 0-3 (CPOL/CPHA), I2C (open-drain, START/STOP, ACK, clock stretching, 7-bit addressing) • **Flow:** Tiny Tapeout IHP CMOS5L template, info.yaml, pin limits (8 in / 8 out / 8 bidir), GitHub Actions hardening, tile/area budget • **Memory:** flip-flop vs SRAM instruction memory area trade-off
Build	• Define the **ISA**: SET/IN/OUT pins, WAIT on pin, JMP/loop counters, delay/cycle-count field, shift registers, TX/RX FIFOs, open-drain control • Choose widths (e.g. 16-bit instr, 32-bit shift reg, 32-64 instr words) and clock-divider design • Block diagram + pin map; write a small **Python assembler** • Set up repo, simulator (cocotb + Icarus/Verilator), CI
Deliverable	• ISA spec (1-2 pages) + block diagram • Assembler working on sample code • Template builds RTL-to-GDS (even with a dummy design)

WEEK 2 (Days 8-14): Core RTL, UART & SPI	
Research	• Cycle-accurate timing: how many clocks per bit for a given baud / SCLK rate • Synchroniser needs for async inputs (metastability, 2-FF) • Hardcaml vs Verilog vs Amaranth/Chisel: pick one and stick to it • Early **Yosys synthesis** area estimation techniques
Build	• RTL: program counter, decoder, instruction memory, shift registers, FIFOs, clock divider, pin mux/direction control, output-enable (open-drain) • Firmware: **UART TX + RX** (configurable baud, parity, stop bits) • Firmware: **SPI master** (all 4 modes, configurable word size), then SPI slave if time allows • cocotb tests with simple protocol reference models; check waveforms (GTKWave) • Run synthesis; record cell count vs the ~24K budget
Deliverable	• Core + UART + SPI passing simulation • First area report; trim ISA/memory if over budget

WEEK 3 (Days 15-21): I2C, Verification & Tape-out Readiness	
Research	• I2C timing tables (standard 100 kHz / fast 400 kHz), clock stretching, repeated START, arbitration (optional) • Verification: constrained-random tests, assertions, formal (SymbiYosys) on FIFO/ISA, coverage • Place-and-route timing closure, clock tree, DRC/LVS errors • Optional: FPGA board for pre-silicon testing
Build	• Firmware: **I2C master** (START/STOP, address, R/W, ACK/NACK, clock stretching, open-drain pins); optional I2C slave • Regression suite: UART loopback, SPI vs reference slave, I2C vs reference slave model • Full **GDS hardening**: fix timing, utilisation, routing; confirm area fits 6x4 • README: ISA, pin map, example programs, how to run tests; tag release
Deliverable	• All 3 protocols pass tests and a clean GDS build • Open-source repo + docs ready; remaining weeks (to 18 Jan) for stretch goals: USB, Ethernet, JTAG

Daily rhythm: ~2 h research/reading + ~2-3 h building. **Tips:** run synthesis from day 1 of RTL, commit often (build in public is allowed), keep the ISA small, and decide by end of Week 1 what you will NOT support. Resources: Tiny Tapeout docs, RP2040 datasheet (PIO chapter), TI PRU docs, NXP I2C spec (UM10204), Jane Street Hardcaml.